

NOTICE TECHNIQUE

Phase 1 : Première approche (Échec et complexe)

Destiné à : Jean-Christophe Dubacq, Micaela Mayero et Haïfa Zargayouna

La méthode de construction

Pour cette phase, nous avons choisi une approche simplifiée et géométrique basée sur notre première compréhension des principes de Voronoi.

Le point le plus important qu'on a noté est que chaque frontière entre les points c'était la médiatrice entre ces deux points.

1^{ère} étape :

Le programme va lire le fichier .txt ligne par ligne et utiliser la fonction « split(« , ») » pour séparer les coordonnées.

```
def lire_points(fichier):  
    points = []  
    f = open(fichier, 'r')  
    for ligne in f:  
        coordonnees = ligne.split(",")  
        x = float(coordonnees[0])  
        y = float(coordonnees[1])  
        points.append((x,y))  
    f.close()  
    return points
```

2^{ème} étape :

Pour chaque paire de points on calcule :

1. Le point du milieu M

```
xm = (point_actuel[0] + point_suivant[0]) / 2  
ym = (point_actuel[1] + point_suivant[1]) / 2
```

2. Le vecteur entre les deux points

```
vx = point_suivant[0] - point_actuel[0]  
vy = point_suivant[1] - point_actuel[1]
```

3. Et le vecteur perpendiculaire

```
vx_perpendiculaire = -vy  
vy_perpendiculaire = vx
```

3^{ème} étape :

C'est la partie la plus complexe du code, l'idée est de trouver tous les points où deux médiatrices se croisent, les intersections. Pour ça on transforme chaque droite en équation cartésienne $Ax + By = C$

Où A est -vy

B est vx

Et $C = A \cdot x_m + B \cdot y_m$

Ensuite on utilise Cramer donc on calcule le déterminant $A_1B_2 - A_2B_1$, si $D = 0$, alors les droites sont parallèles et on ignore, sinon on trouve x et y avec les formules de Cramer.

Les choix

Le code est divisé en plusieurs fonctions :

- lire_points : S'occupe de récupérer les coordonnées dans fichier.txt
- calculer_intersections : Qui va être purement mathématique
- calculer_médiatrices : La fonction principale qui contient une double boucle for pour comparer tous les points entre eux.

On choisit aussi de calculer toutes les intersections possibles entre les médiatrices. C'est une méthode qui consomme des ressources mais ça permet d'éviter d'utiliser des algorithmes trop complexes qu'on n'avait pas compris.

Complexité : $O(n^2)$

Les limites et l'analyse

Au cours du développement, on a tenté de coder une fonction filtrer_intersections. Le but était de garder que les points verts qui sont vraiment des intersections de Voronoi (ceux qui sont à égale distance des trois points noirs les plus proches)

Mais on a abandonné cette piste pour plusieurs raisons :

- Complexité : En codant et en réfléchissant à la suite, c'était devenu beaucoup plus compliqué que prévu notamment pour reconstruire les zones. Avec le temps disponible et sans utilisation d'IA, c'était devenu difficile d'avancer.
- On s'est aussi dit que si une zone de Voronoi était grande, elle n'allait peut-être pas être bien construite à cause du cadre de 500x500, et donc plusieurs problèmes se rajoutaient au problème existant.
- En termes de performance, la double boucle imbriquée était aussi un handicap.